

DM548: Computer Architecture & System Programming DM588: Computer Architecture

Fall 2026

Computer Arithmetic

- Number Systems
- Computer Arithmetic
- Integers
 - Representations
 - Addition and Subtraction
 - Multiplication
 - Division
- Floating-Point Numbers
 - Representation
 - Arithmetic
- Geometric View on Number Representations

Number Systems

- ▶ “There are 10 types of people: those who understand binary and those who don’t.”
- ▶ Normally we are using a **positional number system** with base 10.
- ▶ We will primarily explore base 2, but also briefly base 8 and 16.
- ▶ A number is a string $\underbrace{\dots a_3 a_2 a_1 a_0}_{\text{integer part}} \cdot \underbrace{a_{-1} a_{-2} \dots}_{\text{fractional part}}$ with a base b .
- ▶ It has the value

$$\sum_i (a_i \cdot b^i)$$

- ▶ Example: 42.6 in base 10, also written as 42.6_{10} means $4 \cdot 10^1 + 2 \cdot 10^0 + 6 \cdot 10^{-1}$
- ▶ Example: $1001.101_2 = 2^3 + 2^0 + 2^{-1} + 2^{-3} = 9.625_{10}$

Converting Between Number Systems

Converting to base 10 is relatively simple (see previous slide).

Converting from base 10 to base b :

- ▶ Every integer N can be written as

$$N = b \cdot N_1 + a_0 \quad \text{with } a_0 < b$$

- ▶ well, so can N_1 :

$$N_1 = b \cdot N_2 + a_1 \quad \text{with } a_1 < b$$

- ▶ Putting it together:

$$\begin{aligned} N &= b \cdot (b \cdot N_2 + a_1) + a_0 \\ &= N_2 \cdot b^2 + a_1 \cdot b^1 + a_0 \cdot b^0 \end{aligned}$$

Converting Between Number Systems

Converting to base 10 is relatively simple (see previous slide).

Converting from base 10 to base b :

- ▶ Every integer N can be written as

$$N = b \cdot N_1 + a_0 \quad \text{with } a_0 < b$$

- ▶ well, so can N_1 :

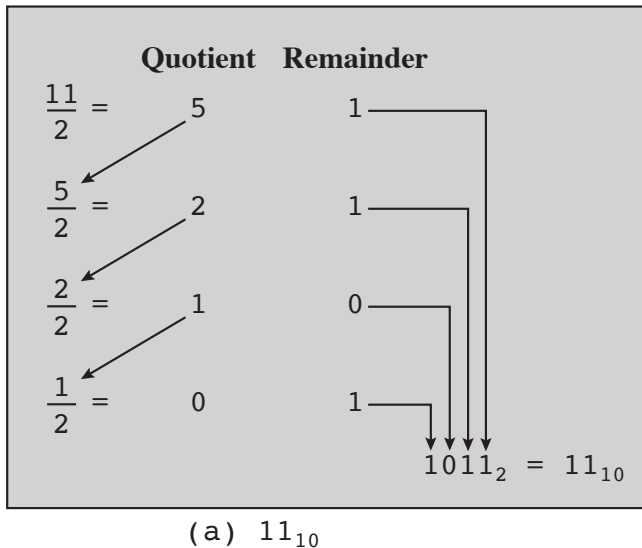
$$N_1 = b \cdot N_2 + a_1 \quad \text{with } a_1 < b$$

- ▶ Putting it together:

$$\begin{aligned} N &= b \cdot (b \cdot N_2 + a_1) + a_0 \\ &= N_2 \cdot b^2 + a_1 \cdot b^1 + a_0 \cdot b^0 \end{aligned}$$

- ▶ If $N_m < b$, then $N_m = b \cdot N_{m+1} + a_m$ gives $N_{m+1} = 0$ and $a_m = N_m$, so stop the expansion.
- ▶ So $N = a_m \cdot b^m + a_{m-1} \cdot b^{m-1} + \dots + a_1 \cdot b^1 + a_0 \cdot b^0$
- ▶ Which is $a_m a_{m-1} \dots a_1 a_0$ in base b

Example



Fractions

- ▶ Convert a number $0 < F < 1$ to $0.a_{-1}a_{-2}\dots$ in base b
- ▶ The number must have the value

$$a_{-1} \cdot b^{-1} + a_{-2} \cdot b^{-2} + \dots$$

iteratively pulling out a factor b^{-1} gives us

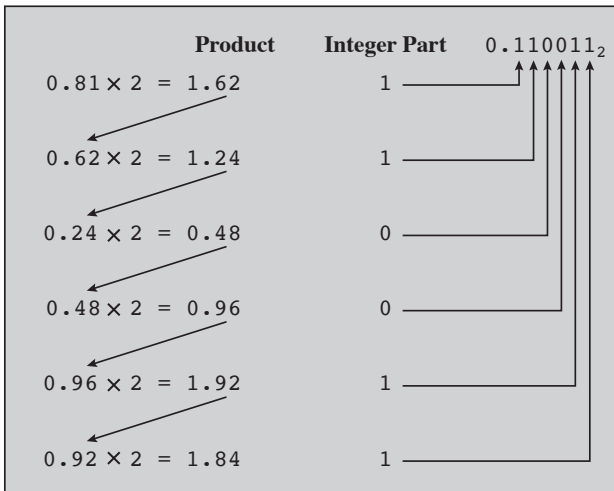
$$b^{-1} \cdot (a_{-1} + b^{-1} \cdot (a_{-2} + b^{-1} \cdot (\dots)))$$

multiplying by b then gives us

$$b \cdot F = a_{-1} + b^{-1} \cdot (a_{-2} + b^{-1} \cdot (\dots))$$

- ▶ So a_{-1} is the integer part of the number $b \cdot F$.
- ▶ Using a scheme similar to that of the integers, we can say that $b \cdot F = a_{-1} + F_1$
- ▶ Calculating $b \cdot F_1$ gives us $a_{-2} + F_2$, etc.

Example



(a) $0.81_{10} = 0.110011_2$ (approximately)

Converting fractions to base 10

- ▶ We can theoretically just use the definition, $\sum_i (a_i \cdot b^i)$:

$$0.110011_2 = \frac{1}{2} + \frac{1}{4} + \frac{1}{32} + \frac{1}{64}$$

- ▶ However, practically your calculator might lose precision while calculating this.
- ▶ Consider scaling first, and then dividing in the end:

$$\begin{aligned} 0.110011_2 &= 110011_2 \cdot 2^{-6} = (1 + 2 + 16 + 32) \cdot 2^{-6} = \frac{51}{64} \\ &= 0.796875_{10} \end{aligned}$$

Common Bases

- ▶ Decimal: base 10
- ▶ Binary: base 2
“bit”: “binary digit”
- ▶ Octal: base 8
 - ▶ Convenient for writing bits: 3 bits $\equiv$ 1 octal digit
- ▶ Hexadecimal: base 16
 - ▶ Very common in computing.
 - ▶ Convenient for writing bits: 4 bits $\equiv$ 1 hex digit
 - 0000 = 0 0100 = 4 1000 = 8 1100 = *C*
 - 0001 = 1 0101 = 5 1001 = 9 1101 = *D*
 - 0010 = 2 0110 = 6 1010 = *A* 1110 = *E*
 - 0011 = 3 0111 = 7 1011 = *B* 1111 = *F*
- ▶ Base 64
 - ▶ E.g., used for encoding binary data and large numbers as ASCII:
 2×26 letters + 10 numbers + 2 characters (e.g., - and _)
 - ▶ [Example](#): Youtube video IDs

- Number Systems
- Computer Arithmetic
- Integers
 - Representations
 - Addition and Subtraction
 - Multiplication
 - Division
- Floating-Point Numbers
 - Representation
 - Arithmetic
- Geometric View on Number Representations

Value and Representation

We need to be careful:

- ▶ Value: the abstract, mathematical thing we want to encode.
- ▶ Representation: the actual bits that encode a value.
- ▶ Sometimes we also use bits when discussing values.
E.g., “the number -101_2 is represented as 011”
E.g., “the number 0101_2 is represented as 1111”
- ▶ We may also use decimal/oct/hex when discussing representations.
E.g., “the representation of 7 is found by adding 8, $7 + 8 = 15$ ”
(Implied: “as how we would convert 15 to base 2”)
E.g., “3.14 is represented as 0x4048f5c3”
(Implied: “as how we convert 0x4048f5c3 to base 2”)

A bit of terminology:

- ▶ MSB: Most Significant Bit (typically the left-most)
- ▶ LSB: Least Significant Bit (typically the right-most)
- ▶ “complementing”/“flipping” bit: change between 0 and 1

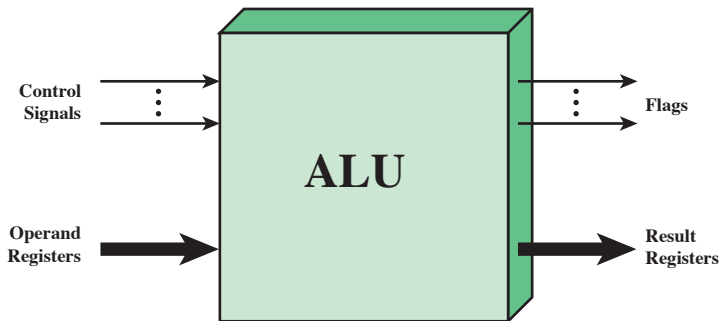
Operations

Where are we actually going with this?

- ▶ We need to design operations that can be implemented in hardware.
- ▶ A bit of a chicken-egg problem:
 - ▶ How does one implement in hardware? Next topic.
 - ▶ Which higher-level operations do we actually need to implement? This topic.
- ▶ Constraints:
 - ▶ We have a limited/fixed amount of space.
 - ▶ Properties of results should be communicated: overflow, zero, negative, positive, parity, ...
 - ▶ We only have 0 and 1.

The ALU

Arithmetic and Logic Unit



- ▶ Control signals: which operation to perform.
- ▶ Flags: properties of the result of the operation.

- Number Systems
- Computer Arithmetic
- Integers
 - Representations
 - Addition and Subtraction
 - Multiplication
 - Division
- Floating-Point Numbers
 - Representation
 - Arithmetic
- Geometric View on Number Representations

Unsigned Integer Representation

- ▶ Assume we have n bits to represent an integer.
- ▶ That means, 2^n different bit-patterns.
- ▶ If we are only representing natural numbers $\mathbb{N}_0$, we can use **unsigned representation**.
- ▶ A bit pattern is interpreted as a number in base 2:
 - ▶ 0000: 0
 - ▶ 0001: 1
 - ▶ 0010: 2
 - ▶ ...
 - ▶ 1111: 15
- ▶ Value range: $[0, 2^n - 1]$

But what about negative numbers?

Sign-Magnitude Representation

- ▶ Reserve the MSB to encode the sign: 0: +, 1: -
- ▶ That leaves $n - 1$ bits for encoding the magnitude (as a base 2 number).
 - ▶ 0 001: +1
 - ▶ 1 001: -1
 - ▶ 0 111: +7
 - ▶ 1 111: -7
- ▶ Value range: $[-(2^{n-1} - 1), 2^{n-1} - 1]$
- ▶ How to compute $-N$? Easy, flip the MSB.
- ▶ How to compare with 0? ... which one of them?

Sign-Magnitude Representation

- ▶ Reserve the MSB to encode the sign: 0: +, 1: -
- ▶ That leaves $n - 1$ bits for encoding the magnitude (as a base 2 number).
 - ▶ 0 001: +1
 - ▶ 1 001: -1
 - ▶ 0 111: +7
 - ▶ 1 111: -7
- ▶ Value range: $[-(2^{n-1} - 1), 2^{n-1} - 1]$
- ▶ How to compute $-N$? Easy, flip the MSB.
- ▶ How to compare with 0? ... which one of them?
- ▶ Major drawback: two representations for 0
 - ▶ 0 000: +0
 - ▶ 1 000: -0
- ▶ Contemplate how often we compare with 0 in programs. How do we implement that with two bit patterns for 0?
- ▶ However, sign-magnitude is relevant for floating-point numbers.

Twos Complement Representation

- ▶ The MSB still **indicates** the sign, but has a negative weight:

$$a_{n-1}a_{n-2}\dots a_1a_0 = -2^{n-1}a_{n-1} + \sum_{i=0}^{n-2} 2^i a_i$$

- ▶ 1000: -8
- ▶ 0000: 0
- ▶ 1001: -7
- ▶ 0001: 1
- ▶ 1111: -1
- ▶ 0111: 7
- ▶ Value range: $[-2^{n-1}, 2^{n-1} - 1]$
- ▶ Only 1 representation for zero!
- ▶ Positive values are encoded directly as base 2 numbers.
- ▶ Doing arithmetic is pretty easy (more later).

Twos Complement Representation

- ▶ The MSB still *indicates* the sign, but has a negative weight:

$$a_{n-1}a_{n-2}\dots a_1a_0 = -2^{n-1}a_{n-1} + \sum_{i=0}^{n-2} 2^i a_i$$

▶ 1000: -8

▶ 0000: 0

▶ 1001: -7

▶ 0001: 1

▶ 1111: -1

▶ 0111: 7

- ▶ Value range: $[-2^{n-1}, 2^{n-1} - 1]$

- ▶ Only 1 representation for zero!

- ▶ Positive values are encoded directly as base 2 numbers.

- ▶ Doing arithmetic is pretty easy (more later).

- ▶ Negating is fairly simple (in reality very easy in the ALU):

1. Complement the number.

2. Add 1.

+18 =	0001 0010
Complement =	1110 1101
Add 1	1
-18 =	1110 1110

Why Does The Negation Algorithm Work?

If A is our non-negative number, and B is the number obtained from the algorithm. Then we should get $A + B = 0$.

Let a_i be a bit in A , and $\overline{a_i}$ its complement:

$$A = -2^{n-1}a_{n-1} + \sum_{i=0}^{n-2} 2^i a_i$$
$$B = 1 - 2^{n-1}\overline{a_{n-1}} + \sum_{i=0}^{n-2} 2^i \overline{a_i}$$

Why Does The Negation Algorithm Work?

If A is our non-negative number, and B is the number obtained from the algorithm. Then we should get $A + B = 0$.

Let a_i be a bit in A , and $\overline{a_i}$ its complement:

$$A = -2^{n-1}a_{n-1} + \sum_{i=0}^{n-2} 2^i a_i$$

$$B = 1 - 2^{n-1}\overline{a_{n-1}} + \sum_{i=0}^{n-2} 2^i \overline{a_i}$$

$$A + B = 1 - 2^{n-1} \cdot (a_{n-1} + \overline{a_{n-1}}) + \sum_{i=0}^{n-2} 2^i \cdot (a_i + \overline{a_i})$$

$$= 1 - 2^{n-1} + \sum_{i=0}^{n-2} 2^i$$

$$= 1 - 2^{n-1} + (2^{n-1} - 1)$$

$$= 0$$

- Number Systems
- Computer Arithmetic
- Integers
 - Representations
 - Addition and Subtraction
 - Multiplication
 - Division
- Floating-Point Numbers
 - Representation
 - Arithmetic
- Geometric View on Number Representations

Addition and Subtraction

- ▶ Addition: add the numbers as if they were base 2 numbers.
- ▶ Subtraction: $A - B = A + (-B)$, so just negate B and do addition.

Examples:

$$5 - 7 = 5 + (-7)$$

5	0101
-7	+1001
-2	1110

$$-4 - 1 = -4 + (-1)$$

-4	1100
-1	+1111
-5	11011

(Discard the carry)

- ▶ Overflow rule:

Overflow occurs if and only if both numbers have the same sign AND the sign changes!

Addition and Subtraction

$$5 - 7 = 5 + (-7)$$

5	0101
-7	+1001
-2	1110

$$-4 - 1 = -4 + (-1)$$

-4	1100
-1	+1111
-5	11011

(Discard the carry)

$$5 + 4$$

5	0101
4	+0100
	1001

$$-7 - 6 = -7 + (-6)$$

-7	1001
-6	+1010
	10011

(Discard the carry)

Overflow occurs if and only if both numbers have the same sign AND the sign changes!

Addition and Subtraction

$$5 - 7 = 5 + (-7)$$

5	0101	Signs are different, no overflow possible.
-7	+1001	
-2	1110	

$$-4 - 1 = -4 + (-1)$$

-4	1100	Sign doesn't change, no overflow.
-1	+1111	
-5	11011	

(Discard the carry)

$$5 + 4$$

5	0101	Overflow! Inputs are positive, result is negative.
4	+0100	
-7	1001	

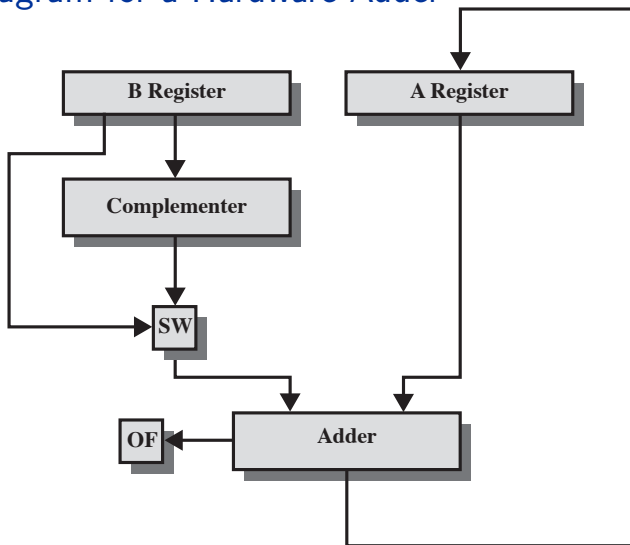
$$-7 - 6 = -7 + (-6)$$

-7	1001	Overflow! Inputs are negative, result is positive.
-6	+1010	
3	10011	

(Discard the carry)

Overflow occurs if and only if both numbers have the same sign AND the sign changes!

Block Diagram for a Hardware Adder



OF = overflow bit

SW = Switch (select addition or subtraction)

- Number Systems
- Computer Arithmetic
- Integers
 - Representations
 - Addition and Subtraction
 - Multiplication**
 - Division
- Floating-Point Numbers
 - Representation
 - Arithmetic
- Geometric View on Number Representations

Multiplication

- ▶ More complicated than addition.
- ▶ Several methods exist.
- ▶ We first look at a basic algorithm.
 - ▶ Essentially like pencil-on-paper multiplication.
 - ▶ It only works for unsigned integer representation.
 - ▶ It is not the most efficient.
- ▶ Then Booth's Algorithm:
 - ▶ Works on twos complement representation.
 - ▶ Requires fewer operations.

From Pencil-and-Paper to Hardware

- ▶ Multiplicand: 11_{10}
- ▶ Multiplier: 13_{10}

From Pencil-and-Paper to Hardware

- ▶ Multiplicand: 11_{10}
- ▶ Multiplier: 13_{10}

Observations:

- ▶ The multiplier has n digits, we do n steps.
- ▶ In base 2 the partial products are either 0 or the multiplicand.
- ▶ The final product has length $2n$ for n -digit inputs.
- ▶ We don't actually need all partial products, just their sum.
- ▶ After each round we don't actually need the just used digit of the multiplier.

From Pencil-and-Paper to Hardware

- ▶ Multiplicand: 11_{10}
- ▶ Multiplier: 13_{10}

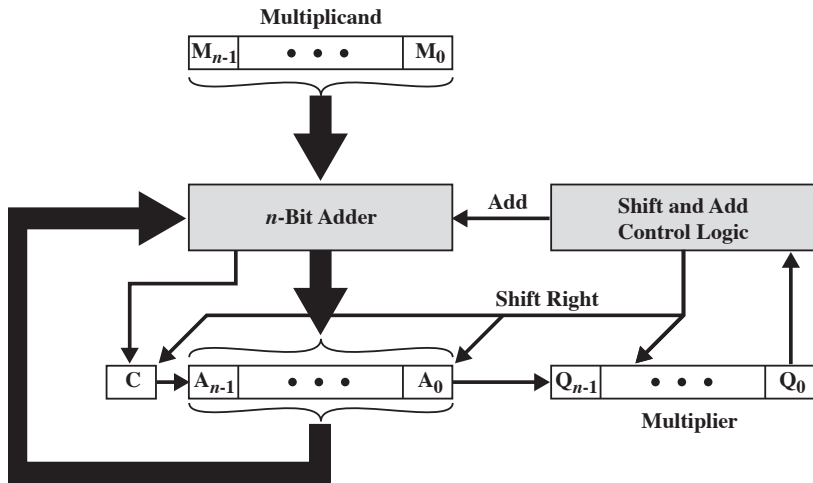
Observations:

- ▶ The multiplier has n digits, we do n steps.
- ▶ In base 2 the partial products are either 0 or the multiplicand.
- ▶ The final product has length $2n$ for n -digit inputs.
- ▶ We don't actually need all partial products, just their sum.
- ▶ After each round we don't actually need the just used digit of the multiplier.

Algorithm outline:

- ▶ Use a $2n$ -bit right-shift register to store the result.
- ▶ Accumulate the partial products in it, and shift for each round.
- ▶ Start with the multiplier in the same register.
- ▶ Each shift destroys the current LSB of the multiplier.

Hardware Implementation



► Total storage: $2n + n + 1$

Example

$$M = 1011$$

Cycle	C	A	Q	
0	0	0000	1101	Initial values

Example

$$M = 1011$$

Cycle	C	A	Q	
0	0	0000	1101	Initial values
1	0	0000 + 1011	1101	Q_0 is 1: add

Example

$$M = 1011$$

Cycle	C	A	Q	
0	0	0000	1101	Initial values
1	0	0000 + 1011	1101	Q_0 is 1: add
	0	1011	1101	$C, A = A + M$

Example

$M = 1011$				
Cycle	C	A	Q	
0	0	0000	1101	Initial values
1	0	0000 + 1011	1101	Q_0 is 1: add
	0	1011	1101	$C, A = A + M$
	0	0101	1110	Shift

Example

$M = 1011$				
Cycle	C	A	Q	
0	0	0000	1101	Initial values
1	0	0000 + 1011	1101	Q_0 is 1: add
	0	1011	1101	$C, A = A + M$
	0	0101	1110	Shift
2	0	0101	1110	Q_0 is 0: no add

Example

$M = 1011$				
Cycle	C	A	Q	
0	0	0000	1101	Initial values
1	0	0000 + 1011	1101	Q_0 is 1: add
	0	1011	1101	$C, A = A + M$
	0	0101	1110	Shift
2	0	0101	1110	Q_0 is 0: no add
	0	0010	1111	Shift

Example

$M = 1011$				
Cycle	C	A	Q	
0	0	0000	1101	Initial values
1	0	0000 + 1011	1101	Q_0 is 1: add
	0	1011	1101	$C, A = A + M$
	0	0101	1110	Shift
2	0	0101	1110	Q_0 is 0: no add
	0	0010	1111	Shift
3	0	0010	1111	Q_0 is 1: add
		+ 1011		

Example

$M = 1011$				
Cycle	C	A	Q	
0	0	0000	1101	Initial values
1	0	0000 + 1011	1101	Q_0 is 1: add
	0	1011	1101	$C, A = A + M$
	0	0101	1110	Shift
2	0	0101	1110	Q_0 is 0: no add
	0	0010	1111	Shift
3	0	0010	1111	Q_0 is 1: add
		+ 1011		
	0	1101	1111	$C, A = A + M$

Example

$$M = 1011$$

Cycle	C	A	Q	
0	0	0000	1101	Initial values
1	0	0000 + 1011	1101	Q_0 is 1: add
	0	1011	1101	$C, A = A + M$
	0	0101	1110	Shift
2	0	0101	1110	Q_0 is 0: no add
	0	0010	1111	Shift
3	0	0010	1111	Q_0 is 1: add
		+ 1011		
	0	1101	1111	$C, A = A + M$
	0	0110	1111	Shift

Example

$M = 1011$				
Cycle	C	A	Q	
0	0	0000	1101	Initial values
1	0	0000 + 1011	1101	Q_0 is 1: add
	0	1011	1101	$C, A = A + M$
	0	0101	1110	Shift
2	0	0101	1110	Q_0 is 0: no add
	0	0010	1111	Shift
3	0	0010	1111	Q_0 is 1: add
		+ 1011		
	0	1101	1111	$C, A = A + M$
	0	0110	1111	Shift
4	0	0110	1111	Q_0 is 1: add
		+ 1011		

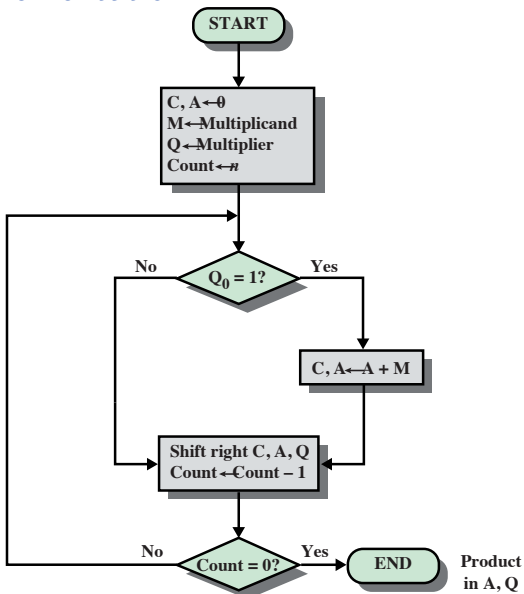
Example

$M = 1011$				
Cycle	C	A	Q	
0	0	0000	1101	Initial values
1	0	0000 + 1011	1101	Q_0 is 1: add
	0	1011	1101	$C, A = A + M$
	0	0101	1110	Shift
2	0	0101	1110	Q_0 is 0: no add
	0	0010	1111	Shift
3	0	0010	1111	Q_0 is 1: add
		+ 1011		
	0	1101	1111	$C, A = A + M$
	0	0110	1111	Shift
4	0	0110	1111	Q_0 is 1: add
		+ 1011		
	1	0001	1111	$C, A = A + M$

Example

$M = 1011$				
Cycle	C	A	Q	
0	0	0000	1101	Initial values
1	0	0000 + 1011	1101	Q_0 is 1: add
	0	1011	1101	$C, A = A + M$
	0	0101	1110	Shift
2	0	0101	1110	Q_0 is 0: no add
	0	0010	1111	Shift
3	0	0010	1111	Q_0 is 1: add
		+ 1011		
	0	1101	1111	$C, A = A + M$
	0	0110	1111	Shift
4	0	0110	1111	Q_0 is 1: add
		+ 1011		
	1	0001	1111	$C, A = A + M$
	0	1000	1111	Shift
Result: 10001111				
Decimal: 143				

Flowchart Implementation



Twos Complement Multiplication

- ▶ For addition (and subtraction) we could just use the method for unsigned integers, with a special overflow rule.
- ▶ For multiplication we can not. Let's try and see why it fails.
- ▶ First consider $-7 \cdot 3$, and let's write the full partial sums:

$$\begin{array}{r} 1001 \cdot 0011 \\ \hline 0000 \text{ } 1001 \\ 0001 \text{ } 0010 \\ 0000 \text{ } 0000 \\ + 0000 \text{ } 0000 \\ \hline 0001 \text{ } 1011 \end{array} = 27_{10} \neq -21_{10}$$

Twos Complement Multiplication

- ▶ For addition (and subtraction) we could just use the method for unsigned integers, with a special overflow rule.
- ▶ For multiplication we can not. Let's try and see why it fails.
- ▶ First consider $-7 \cdot 3$, and let's write the full partial sums:

$$\begin{array}{r} 1001 \cdot 0011 \\ \hline 0000\ 1001 \\ 0001\ 0010 \\ 0000\ 0000 \\ +\ 0000\ 0000 \\ \hline 0001\ 1011 \end{array} = 27_{10} \neq -21_{10}$$

- ▶ We should have added a bunch of negative numbers.
- ▶ We convert 4-bit numbers to 8-bit numbers,
- ▶ but the zeroes in front makes all the partial products positive.

Sign-Extension

- ▶ How do we actually convert an n -bit twos complement number to $n + k$ bits?
- ▶ This is called **sign-extension**.
- ▶ For sign-magnitude: the MSB is the sign, so inject zeroes
 - ▶ $a_{n-1}a_{n-2} \dots a_1a_0$ becomes
 - ▶ $a_{n_1}00 \dots 0a_{n-2} \dots a_1a_0$
 - ▶ E.g., 1100 $\rightarrow$ 10000100

Sign-Extension

- ▶ How do we actually convert an n -bit twos complement number to $n + k$ bits?
- ▶ This is called **sign-extension**.
- ▶ For sign-magnitude: the MSB is the sign, so inject zeroes
 - ▶ $a_{n-1}a_{n-2} \dots a_1a_0$ becomes
 - ▶ $a_{n-1}00 \dots 0a_{n-2} \dots a_1a_0$
 - ▶ E.g., 1100 $\rightarrow$ 1000 0100
- ▶ For twos complement: copy the MSB
 - ▶ E.g., 0100 $\rightarrow$ 0000 0100
 - ▶ E.g., 1100 $\rightarrow$ 1111 1100

Sign-Extension

- ▶ How do we actually convert an n -bit twos complement number to $n + k$ bits?
- ▶ This is called **sign-extension**.
- ▶ For sign-magnitude: the MSB is the sign, so inject zeroes
 - ▶ $a_{n-1}a_{n-2}\dots a_1a_0$ becomes
 - ▶ $a_{n-1}00\dots 0a_{n-2}\dots a_1a_0$
 - ▶ E.g., 1100 $\rightarrow$ 1000 0100
- ▶ For twos complement: copy the MSB
 - ▶ E.g., 0100 $\rightarrow$ 0000 0100
 - ▶ E.g., 1100 $\rightarrow$ 1111 1100
- ▶ Why does it work? Consider $m > n$ and a negative number A

$$A = -2^{n-1}a_{n-1} + \sum_{i=0}^{n-2} 2^i a_i$$

$$A = -2^{m-1}a_{m-1} + \sum_{i=0}^{m-2} 2^i a_i$$

Try setting them equal to each other (and see the book).

Back to Multiplication

- ▶ Using sign-extension in the partial products?
- ▶ Now it sometimes work. Consider again $-7 \cdot 3$:

$$\begin{array}{r} 1001 \cdot 0011 \\ \hline 1111 \text{ } 1001 = (-7) \cdot 2^0 \\ 1111 \text{ } 0010 = (-7) \cdot 2^1 = -14 \\ 0000 \text{ } 0000 \\ + 0000 \text{ } 0000 \\ \hline 1110 \text{ } 1011 = -21_{10} \end{array}$$

Back to Multiplication

- ▶ Using sign-extension in the partial products?
- ▶ Now it sometimes work. Consider again $-7 \cdot 3$:

$$\begin{array}{r} 1001 \cdot 0011 \\ \hline 1111 \text{ } 1001 = (-7) \cdot 2^0 \\ 1111 \text{ } 0010 = (-7) \cdot 2^1 = -14 \\ 0000 \text{ } 0000 \\ + 0000 \text{ } 0000 \\ \hline 1110 \text{ } 1011 = -21_{10} \end{array}$$

- ▶ But not always: $3 \cdot (-7)$

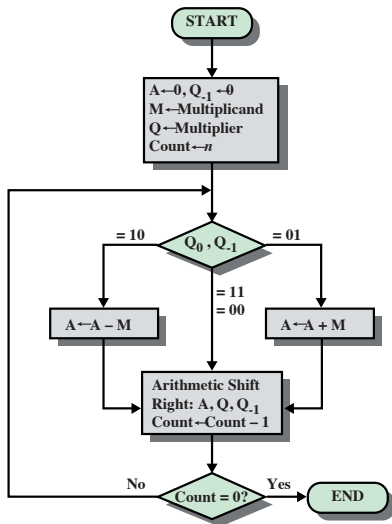
$$\begin{array}{r} 0011 \cdot 1001 \\ \hline 0000 \text{ } 0011 \\ 0000 \text{ } 0000 \\ 0000 \text{ } 0000 \\ + 0001 \text{ } 1000 \\ \hline 0001 \text{ } 1011 = 27_{10} \end{array}$$

What are we actually doing?

- ▶ We use repeated addition to perform multiplication.
- ▶ To perform $M \cdot Q$ we add M to 0 a number of times.
- ▶ Q is not used as a value, but as a sequence of digits.
- ▶ The value and position of the digit determines the partial sum.
- ▶ In binary, each 1 translates directly to an addition of M .
- ▶ But in twos complement the digits in a negative number is merely a **representation** of the number.

A Solution: Booth's Algorithm

- ▶ As before: multiplicand in M , multiplier in Q .
- ▶ We remove the carry register,
- ▶ but add 1-bit register Q_{-1} on the right side.
- ▶ Shift register: $A : Q : Q_{-1}$
- ▶ The algorithm may now add or subtract M from A :
 - ▶ $Q_0Q_{-1} = 10$: subtract
 - ▶ $Q_0Q_{-1} = 01$: add
 - ▶ $Q_0Q_{-1} = 00$ or 11 : only shift
- ▶ The shift is an **arithmetic** shift, i.e., fill in with the MSB, indicating the sign.



Booth's Algorithm: Example

$$M = 0111$$

$$-M = 1001$$

Cycle	A	Q	Q_{-1}	
0	0000	1101	0	Initial values

- ▶ $Q_0Q_{-1} = 10$: subtract
- ▶ $Q_0Q_{-1} = 01$: add
- ▶ $Q_0Q_{-1} = 00$ or 11 :
only shift

Booth's Algorithm: Example

$$M = 0111$$

$$-M = 1001$$

Cycle	A	Q	Q_{-1}	
0	0000	1101	0	Initial values
1	0000 + 1001	1101	0	$Q_0Q_{-1} = 10$

- ▶ $Q_0Q_{-1} = 10$: subtract
- ▶ $Q_0Q_{-1} = 01$: add
- ▶ $Q_0Q_{-1} = 00$ or 11 :
only shift

Booth's Algorithm: Example

$$M = 0111$$

$$-M = 1001$$

Cycle	A	Q	Q ₋₁	
0	0000	1101	0	Initial values
1	0000 + 1001	1101	0	$Q_0Q_{-1} = 10$
	1001	1101	0	$A = A - M$

- ▶ $Q_0Q_{-1} = 10$: subtract
- ▶ $Q_0Q_{-1} = 01$: add
- ▶ $Q_0Q_{-1} = 00$ or 11 :
only shift

Booth's Algorithm: Example

$$M = 0111$$

$$-M = 1001$$

Cycle	A	Q	Q ₋₁	
0	0000	1101	0	Initial values
1	0000 + 1001	1101	0	$Q_0Q_{-1} = 10$
	1001 1100	1101 1110	0 1	$A = A - M$ Shift

- ▶ $Q_0Q_{-1} = 10$: subtract
- ▶ $Q_0Q_{-1} = 01$: add
- ▶ $Q_0Q_{-1} = 00$ or 11 :
only shift

Booth's Algorithm: Example

$$M = 0111$$

$$-M = 1001$$

Cycle	A	Q	Q ₋₁	
0	0000	1101	0	Initial values
1	0000 + 1001	1101	0	$Q_0Q_{-1} = 10$
	1001 1100	1101 1110	0 1	$A = A - M$ Shift
2	1100 + 0111	1110	1	$Q_0Q_{-1} = 01$

► $Q_0Q_{-1} = 10$: subtract

► $Q_0Q_{-1} = 01$: add

► $Q_0Q_{-1} = 00$ or 11 :
only shift

Booth's Algorithm: Example

$$M = 0111$$

$$-M = 1001$$

► $Q_0Q_{-1} = 10$: subtract

► $Q_0Q_{-1} = 01$: add

► $Q_0Q_{-1} = 00$ or 11 :
only shift

Cycle	A	Q	Q_{-1}	
0	0000	1101	0	Initial values
1	0000 + 1001	1101	0	$Q_0Q_{-1} = 10$
	1001 1100	1101 1110	0 1	$A = A - M$ Shift
2	1100 + 0111	1110	1	$Q_0Q_{-1} = 01$
	0011	1110	1	$A = A + M$

Booth's Algorithm: Example

$$M = 0111$$

$$-M = 1001$$

► $Q_0Q_{-1} = 10$: subtract

► $Q_0Q_{-1} = 01$: add

► $Q_0Q_{-1} = 00$ or 11 :
only shift

Cycle	A	Q	Q_{-1}	
0	0000	1101	0	Initial values
1	0000 + 1001	1101	0	$Q_0Q_{-1} = 10$
	1001 1100	1101 1110	0 1	$A = A - M$ Shift
2	1100 + 0111	1110	1	$Q_0Q_{-1} = 01$
	0011 0001	1110 1111	1 0	$A = A + M$ Shift

Booth's Algorithm: Example

$$M = 0111$$

$$-M = 1001$$

► $Q_0Q_{-1} = 10$: subtract

► $Q_0Q_{-1} = 01$: add

► $Q_0Q_{-1} = 00$ or 11 :
only shift

Cycle	A	Q	Q_{-1}	
0	0000	1101	0	Initial values
1	0000 + 1001	1101	0	$Q_0Q_{-1} = 10$
	1001 1100	1101 1110	0 1	$A = A - M$ Shift
2	1100 + 0111	1110	1	$Q_0Q_{-1} = 01$
	0011 0001	1110 1111	1 0	$A = A + M$ Shift
3	0001 + 1001	1111	0	$Q_0Q_{-1} = 10$

Booth's Algorithm: Example

$$M = 0111$$

$$-M = 1001$$

► $Q_0Q_{-1} = 10$: subtract

► $Q_0Q_{-1} = 01$: add

► $Q_0Q_{-1} = 00$ or 11 :
only shift

Cycle	A	Q	Q_{-1}	
0	0000	1101	0	Initial values
1	0000 + 1001	1101	0	$Q_0Q_{-1} = 10$
	1001 1100	1101 1110	0 1	$A = A - M$ Shift
2	1100 + 0111	1110	1	$Q_0Q_{-1} = 01$
	0011 0001	1110 1111	1 0	$A = A + M$ Shift
3	0001 + 1001	1111	0	$Q_0Q_{-1} = 10$
	1010	1111	0	$A = A - M$

Booth's Algorithm: Example

$$M = 0111$$

$$-M = 1001$$

► $Q_0Q_{-1} = 10$: subtract

► $Q_0Q_{-1} = 01$: add

► $Q_0Q_{-1} = 00$ or 11 :
only shift

Cycle	A	Q	Q_{-1}	
0	0000	1101	0	Initial values
1	0000 + 1001	1101	0	$Q_0Q_{-1} = 10$
	1001 1100	1101 1110	0 1	$A = A - M$ Shift
2	1100 + 0111	1110	1	$Q_0Q_{-1} = 01$
	0011 0001	1110 1111	1 0	$A = A + M$ Shift
3	0001 + 1001	1111	0	$Q_0Q_{-1} = 10$
	1010 1101	1111 0111	0 1	$A = A - M$ Shift

Booth's Algorithm: Example

$$M = 0111$$

$$-M = 1001$$

► $Q_0Q_{-1} = 10$: subtract

► $Q_0Q_{-1} = 01$: add

► $Q_0Q_{-1} = 00$ or 11 :
only shift

Cycle	A	Q	Q_{-1}	
0	0000	1101	0	Initial values
1	0000 + 1001	1101	0	$Q_0Q_{-1} = 10$
	1001 1100	1101 1110	0 1	$A = A - M$ Shift
2	1100 + 0111	1110	1	$Q_0Q_{-1} = 01$
	0011 0001	1110 1111	1 0	$A = A + M$ Shift
3	0001 + 1001	1111	0	$Q_0Q_{-1} = 10$
	1010 1101	1111 0111	0 1	$A = A - M$ Shift
4	1101	0111	1	$Q_0Q_{-1} = 11$

Booth's Algorithm: Example

$$M = 0111$$

$$-M = 1001$$

► $Q_0Q_{-1} = 10$: subtract

► $Q_0Q_{-1} = 01$: add

► $Q_0Q_{-1} = 00$ or 11 :
only shift

Cycle	A	Q	Q_{-1}	
0	0000	1101	0	Initial values
1	0000 + 1001	1101	0	$Q_0Q_{-1} = 10$
	1001 1100	1101 1110	0 1	$A = A - M$ Shift
2	1100 + 0111	1110	1	$Q_0Q_{-1} = 01$
	0011 0001	1110 1111	1 0	$A = A + M$ Shift
3	0001 + 1001	1111	0	$Q_0Q_{-1} = 10$
	1010 1101	1111 0111	0 1	$A = A - M$ Shift
4	1101 1110	0111 1011	1 1	$Q_0Q_{-1} = 11$ Shift

Result: 1110 1011

Decimal: -21

Why does it work?

- ▶ Let's consider first positive multipliers,
- ▶ and just those with one block of 1's.
- ▶ For example:

$$M \cdot 0001\ 1110 = M \cdot (2^4 + 2^3 + 2^2 + 2^1) = M \cdot 30$$

- ▶ A generic block of k 1's, from position n down-wards, has the value:
$$2^n + 2^{n-1} + \dots + 2^{n-k}$$

Why does it work?

- ▶ Let's consider first positive multipliers,
- ▶ and just those with one block of 1's.
- ▶ For example:

$$M \cdot 0001\ 1110 = M \cdot (2^4 + 2^3 + 2^2 + 2^1) = M \cdot 30$$

- ▶ A generic block of k 1's, from position n down-wards, has the value:
$$2^n + 2^{n-1} + \dots + 2^{n-k} = \sum_{i=n-k}^n 2^i = 2^{n+1} - 2^{n-k}$$
- ▶ For example:

$$M \cdot 0001\ 1110 = M \cdot (2^4 + 2^3 + 2^2 + 2^1) = 2^5 - 2^1 = 32 - 2 = 30$$

Why does it work?

- ▶ Let's consider first positive multipliers,
- ▶ and just those with one block of 1's.
- ▶ For example:

$$M \cdot 0001\ 1110 = M \cdot (2^4 + 2^3 + 2^2 + 2^1) = M \cdot 30$$

- ▶ A generic block of k 1's, from position n down-wards, has the value:
$$2^n + 2^{n-1} + \dots + 2^{n-k} = \sum_{i=n-k}^n 2^i = 2^{n+1} - 2^{n-k}$$

- ▶ For example:

$$M \cdot 0001\ 1110 = M \cdot (2^4 + 2^3 + 2^2 + 2^1) = 2^5 - 2^1 = 32 - 2 = 30$$

- ▶ It also works for a block of size 1:

$$M \cdot 2^i = M \cdot (2^{i+1} - 2^i)$$

Why does it work?

- ▶ Let's consider first positive multipliers,
- ▶ and just those with one block of 1's.
- ▶ For example:

$$M \cdot 0001\ 1110 = M \cdot (2^4 + 2^3 + 2^2 + 2^1) = M \cdot 30$$

- ▶ A generic block of k 1's, from position n down-wards, has the value:
$$2^n + 2^{n-1} + \dots + 2^{n-k} = \sum_{i=n-k}^n 2^i = 2^{n+1} - 2^{n-k}$$

- ▶ For example:

$$M \cdot 0001\ 1110 = M \cdot (2^4 + 2^3 + 2^2 + 2^1) = 2^5 - 2^1 = 32 - 2 = 30$$

- ▶ It also works for a block of size 1:

$$M \cdot 2^i = M \cdot (2^{i+1} - 2^i)$$

- ▶ Booth's Algorithm:

- ▶ When finding the right edge of a block (10): subtract
- ▶ When finding the left edge of a block (01): add
- ▶ Otherwise, only do shifts.
- ▶ This means the algorithm usually is more efficient than the one for unsigned integers.

How about negative multipliers?

- ▶ A negative number has 1 as the MSB.
- ▶ So the last block (the left-most) has no left edge.
- ▶ Even for $-1 = 111 \dots 1$ there is a right edge:
we add the Q_{-1} register initialized to 0.
- ▶ So we will have a subtraction without an addition.

How about negative multipliers?

- ▶ A negative number has 1 as the MSB.
- ▶ So the last block (the left-most) has no left edge.
- ▶ Even for $-1 = 111 \dots 1$ there is a right edge:
we add the Q_{-1} register initialized to 0.
- ▶ So we will have a subtraction without an addition.
- ▶ See the book for the full proof.
- ▶ Example:
$$M \cdot (-6) = M \cdot 1111\ 1010 = M \cdot (-2^7 + 2^5 + 2^4 + 2^3 + 2^1)$$
- ▶ Going LSB-to-MSB (right-to-left):
 - ▶ Block openings: 2^1 and 2^3
 - ▶ Block closings: 2^2
- ▶ Using Booth's algorithm:

$$\begin{aligned} M \cdot 1111\ 1010 &= M \cdot ((-2^3) + (2^2 - 2^1)) \\ &= M \cdot (-8 + 4 - 2) = M \cdot (-6) \end{aligned}$$

- Number Systems
- Computer Arithmetic
- Integers
 - Representations
 - Addition and Subtraction
 - Multiplication
 - Division
- Floating-Point Numbers
 - Representation
 - Arithmetic
- Geometric View on Number Representations

Division (of unsigned integers)

- ▶ Do you remember the pencil-and-paper algorithm?

- ▶ Example: $\frac{147}{11} = 13$, with remainder 4

- ▶ Dividend: 147

- ▶ Divisor: 11

- ▶ Quotient: 13

- ▶ Remainder: 4

(10010011₂)

(1011₂)

(1101₂)

(0100₂)

Quotient

147 : 11

Division (of unsigned integers)

- ▶ Do you remember the pencil-and-paper algorithm?

- ▶ Example: $\frac{147}{11} = 13$, with remainder 4

- ▶ Dividend: 147

- ▶ Divisor: 11

- ▶ Quotient: 13

- ▶ Remainder: 4

(10010011₂)

(1011₂)

(1101₂)

(0100₂)

1 Quotient

147 : 11
11

 37

Division (of unsigned integers)

- ▶ Do you remember the pencil-and-paper algorithm?

- ▶ Example: $\frac{147}{11} = 13$, with remainder 4

- ▶ Dividend: 147

- ▶ Divisor: 11

- ▶ Quotient: 13

- ▶ Remainder: 4

(10010011₂)

(1011₂)

(1101₂)

(0100₂)

1 Quotient

147 : 11
11

3

Division (of unsigned integers)

- ▶ Do you remember the pencil-and-paper algorithm?

- ▶ Example: $\frac{147}{11} = 13$, with remainder 4

- ▶ Dividend: 147

(10010011_2)

- ▶ Divisor: 11

(1011_2)

- ▶ Quotient: 13

(1101_2)

- ▶ Remainder: 4

(0100_2)

1	Quotient
147 : 11	
11	
37	Partial remainder

Division (of unsigned integers)

- ▶ Do you remember the pencil-and-paper algorithm?

- ▶ Example: $\frac{147}{11} = 13$, with remainder 4

- ▶ Dividend: 147

- ▶ Divisor: 11

- ▶ Quotient: 13

- ▶ Remainder: 4

(10010011₂)

(1011₂)

(1101₂)

(0100₂)

13	Quotient
147 : 11	
11	
37	Partial remainder
33	

Division (of unsigned integers)

- ▶ Do you remember the pencil-and-paper algorithm?

- ▶ Example: $\frac{147}{11} = 13$, with remainder 4

- ▶ Dividend: 147

- ▶ Divisor: 11

- ▶ Quotient: 13

- ▶ Remainder: 4

(10010011₂)

(1011₂)

(1101₂)

(0100₂)

13	Quotient
147 : 11	
11	
37	Partial remainder
33	
4	(Partial) remainder

Division in Binary

- ▶ Dividend: 147
- ▶ Divisor: 11
- ▶ Quotient: 13
- ▶ Remainder: 4

(10010011_2)
 (1011_2)
 (1101_2)
 (0100_2)

Quotient

10010011 : 1011

Division in Binary

- ▶ Dividend: 147
- ▶ Divisor: 11
- ▶ Quotient: 13
- ▶ Remainder: 4

(10010011_2)
 (1011_2)
 (1101_2)
 (0100_2)

1 Quotient

10010011 : 1011
 1011

Division in Binary

- ▶ Dividend: 147
- ▶ Divisor: 11
- ▶ Quotient: 13
- ▶ Remainder: 4

(10010011_2)
 (1011_2)
 (1101_2)
 (0100_2)

1 Quotient

10010011 : 1011
 1011

 111

Division in Binary

- ▶ Dividend: 147
- ▶ Divisor: 11
- ▶ Quotient: 13
- ▶ Remainder: 4

(10010011_2)
 (1011_2)
 (1101_2)
 (0100_2)

1	Quotient
10010011 : 1011	
1011	
1110	Partial remainder

Division in Binary

- ▶ Dividend: 147
- ▶ Divisor: 11
- ▶ Quotient: 13
- ▶ Remainder: 4

(10010011_2)
 (1011_2)
 (1101_2)
 (0100_2)

11	Quotient
10010011 : 1011	
1011	
1110	Partial remainder
1011	

Division in Binary

- ▶ Dividend: 147
- ▶ Divisor: 11
- ▶ Quotient: 13
- ▶ Remainder: 4

(10010011_2)
 (1011_2)
 (1101_2)
 (0100_2)

11	Quotient
10010011 : 1011	
1011	
1110	Partial remainder
1011	
11	

Division in Binary

- ▶ Dividend: 147
- ▶ Divisor: 11
- ▶ Quotient: 13
- ▶ Remainder: 4

(10010011_2)
 (1011_2)
 (1101_2)
 (0100_2)

11	Quotient
10010011 : 1011	
1011	
1110	Partial remainder
1011	
111	

Division in Binary

- ▶ Dividend: 147
- ▶ Divisor: 11
- ▶ Quotient: 13
- ▶ Remainder: 4

(10010011_2)
 (1011_2)
 (1101_2)
 (0100_2)

110	Quotient
10010011 : 1011	
1011	
1110	Partial remainder
1011	
1111	Partial remainder

Division in Binary

- ▶ Dividend: 147
- ▶ Divisor: 11
- ▶ Quotient: 13
- ▶ Remainder: 4

(10010011_2)
 (1011_2)
 (1101_2)
 (0100_2)

1101	Quotient
10010011 : 1011	
1011	
1110	Partial remainder
1011	
1111	Partial remainder
1011	

Division in Binary

- ▶ Dividend: 147
- ▶ Divisor: 11
- ▶ Quotient: 13
- ▶ Remainder: 4

(10010011_2)
 (1011_2)
 (1101_2)
 (0100_2)

1101	Quotient
10010011 : 1011	
1011	
1110	Partial remainder
1011	
1111	Partial remainder
1011	
100	(Partial) remainder

Division in Binary

- ▶ Dividend: 147 (10010011_2)
- ▶ Divisor: 11 (1011_2)
- ▶ Quotient: 13 (1101_2)
- ▶ Remainder: 4 (0100_2)

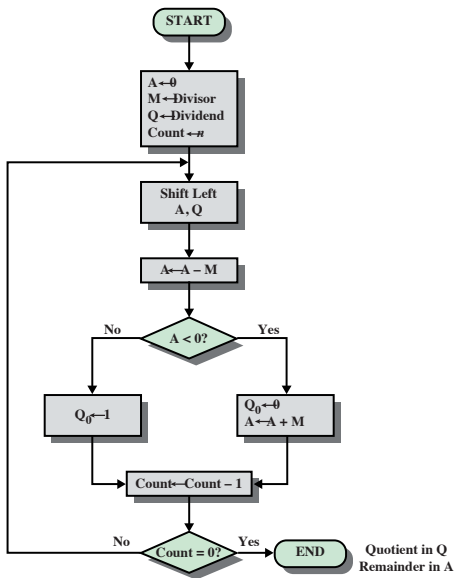
$\begin{array}{r} 1101 \\ \hline 10010011 : 1011 \\ \underline{1011} \\ 1110 \\ \underline{1011} \\ 1111 \\ \underline{1011} \\ 100 \end{array}$	Quotient
$\begin{array}{r} 1110 \\ \underline{1011} \\ 1111 \\ \underline{1011} \\ 100 \end{array}$	Partial remainder
$\begin{array}{r} 1111 \\ \underline{1011} \\ 100 \end{array}$	Partial remainder
$\begin{array}{r} 100 \end{array}$	(Partial) remainder

Observations:

- ▶ The multiples are either 0 or 1,
- ▶ so we can test with a simple subtraction.
- ▶ Only a shrinking suffix of the divisor is needed,
- ▶ and the quotient is found 1 bit at a time.

Division Algorithm

- ▶ Divide n -bit non-negative numbers in two's complement representation.
- ▶ Use a $2n$ -bit shift register:
 $A : Q$
- ▶ Fill A with 0.
- ▶ Use A to hold the partial remainder.
- ▶ Set quotient bits in Q_0 after left-shifts.
- ▶ We do subtraction by addition in two's complement.



Example: $\frac{7}{3}$

Cycle	A	Q	
0	0000	0111	Initial values

$$M = 0011$$

$$-M = 1101$$

Example: $\frac{7}{3}$

Cycle	A	Q	
0	0000	0111	Initial values
1	0000	1110	Shift

$$M = 0011$$

$$-M = 1101$$

Example: $\frac{7}{3}$

Cycle	A	Q	
0	0000	0111	Initial values
1	0000 + 1101 1101	1110	Shift $A = A + (-M)$ Negative, can not divide

$$M = 0011$$

$$-M = 1101$$

Example: $\frac{7}{3}$

Cycle	A	Q	
0	0000	0111	Initial values
1	0000 + 1101	1110	Shift $A = A + (-M)$
	1101 0000	1110	Negative, can not divide Restore A, set $Q_0 = 0$

$$M = 0011$$

$$-M = 1101$$

Example: $\frac{7}{3}$

Cycle	A	Q	
0	0000	0111	Initial values
1	0000 + 1101 1101 0000	1110	Shift $A = A + (-M)$ Negative, can not divide Restore A, set $Q_0 = 0$
2	0001	1100	Shift

$$M = 0011$$

$$-M = 1101$$

Example: $\frac{7}{3}$

Cycle	A	Q	
0	0000	0111	Initial values
1	0000 + 1101 1101 0000	1110	Shift $A = A + (-M)$ Negative, can not divide Restore A, set $Q_0 = 0$
2	0001 + 1101 1110	1100	Shift $A = A + (-M)$ Negative, can not divide

$$M = 0011$$

$$-M = 1101$$

Example: $\frac{7}{3}$

$$M = 0011$$

$$-M = 1101$$

Cycle	A	Q	
0	0000	0111	Initial values
1	0000 + 1101 1101 0000	1110	Shift $A = A + (-M)$ Negative, can not divide Restore A, set $Q_0 = 0$
2	0001 + 1101 1110 0001	1100	Shift $A = A + (-M)$ Negative, can not divide Restore A, set $Q_0 = 0$

Example: $\frac{7}{3}$

$$M = 0011$$

$$-M = 1101$$

Cycle	A	Q	
0	0000	0111	Initial values
1	0000	1110	Shift
	+ 1101		$A = A + (-M)$
	1101		Negative, can not divide
	0000	1110	Restore A, set $Q_0 = 0$
2	0001	1100	Shift
	+ 1101		$A = A + (-M)$
	1110		Negative, can not divide
	0001	1100	Restore A, set $Q_0 = 0$
3	0011	1000	Shift

Example: $\frac{7}{3}$

$$M = 0011$$

$$-M = 1101$$

Cycle	A	Q	
0	0000	0111	Initial values
1	0000 + 1101 1101 0000	1110	Shift $A = A + (-M)$ Negative, can not divide Restore A, set $Q_0 = 0$
2	0001 + 1101 1110 0001	1100	Shift $A = A + (-M)$ Negative, can not divide Restore A, set $Q_0 = 0$
3	0011 + 1101 0000	1000	Shift $A = A + (-M)$ Non-negative, divides

Example: $\frac{7}{3}$

$$M = 0011$$

$$-M = 1101$$

Cycle	A	Q	
0	0000	0111	Initial values
1	0000 + 1101 1101 0000	1110	Shift $A = A + (-M)$ Negative, can not divide Restore A, set $Q_0 = 0$
2	0001 + 1101 1110 0001	1100	Shift $A = A + (-M)$ Negative, can not divide Restore A, set $Q_0 = 0$
3	0011 + 1101 0000 0000	1000 1001	Shift $A = A + (-M)$ Non-negative, divides Keep A, set $Q_0 = 1$

Example: $\frac{7}{3}$

$$M = 0011$$

$$-M = 1101$$

Cycle	A	Q	
0	0000	0111	Initial values
1	0000 + 1101 1101 0000	1110	Shift $A = A + (-M)$ Negative, can not divide Restore A, set $Q_0 = 0$
2	0001 + 1101 1110 0001	1100	Shift $A = A + (-M)$ Negative, can not divide Restore A, set $Q_0 = 0$
3	0011 + 1101 0000 0000	1000	Shift $A = A + (-M)$ Non-negative, divides Keep A, set $Q_0 = 1$
4	0001	0010	Shift

Example: $\frac{7}{3}$

$$M = 0011$$

$$-M = 1101$$

Cycle	A	Q	
0	0000	0111	Initial values
1	0000 + 1101 1101 0000	1110	Shift $A = A + (-M)$ Negative, can not divide Restore A, set $Q_0 = 0$
2	0001 + 1101 1110 0001	1100	Shift $A = A + (-M)$ Negative, can not divide Restore A, set $Q_0 = 0$
3	0011 + 1101 0000 0000	1000	Shift $A = A + (-M)$ Non-negative, divides Keep A, set $Q_0 = 1$
4	0001 + 1101 1110	0010	Shift $A = A + (-M)$ Negative, can not divide

Example: $\frac{7}{3}$

$$M = 0011$$

$$-M = 1101$$

Cycle	A	Q	
0	0000	0111	Initial values
1	0000 + 1101 1101 0000	1110	Shift $A = A + (-M)$ Negative, can not divide Restore A, set $Q_0 = 0$
2	0001 + 1101 1110 0001	1100	Shift $A = A + (-M)$ Negative, can not divide Restore A, set $Q_0 = 0$
3	0011 + 1101 0000 0000	1000	Shift $A = A + (-M)$ Non-negative, divides Keep A, set $Q_0 = 1$
4	0001 + 1101 1110 0001	0010	Shift $A = A + (-M)$ Negative, can not divide Restore A, set $Q_0 = 0$
Quotient: 0010 (decimal: 2)			
Remainder: 0001 (decimal: 1)			

Negative Numbers

- ▶ This approach could be extended to work with negative numbers.
- ▶ In practice, it is calculated with positive numbers,
- ▶ and in the end, the signs of the result and the remainder are assigned: for D/V yielding Q and remainder R we would like $D = Q \cdot V + R$

$$D = 7 \quad V = 3 \quad \Rightarrow \quad Q = 2 \quad R = 1$$

$$D = 7 \quad V = -3 \quad \Rightarrow \quad Q = -2 \quad R = 1$$

$$D = -7 \quad V = 3 \quad \Rightarrow \quad Q = -2 \quad R = -1$$

$$D = -7 \quad V = -3 \quad \Rightarrow \quad Q = 2 \quad R = -1$$

- ▶ So, $\text{sign}(Q) = \text{sign}(D) \cdot \text{sign}(V)$ and $\text{sign}(R) = \text{sign}(D)$.

- Number Systems
- Computer Arithmetic
- Integers
 - Representations
 - Addition and Subtraction
 - Multiplication
 - Division
- Floating-Point Numbers
 - Representation
 - Arithmetic
- Geometric View on Number Representations

How about fractions?

- ▶ Our integer representations are special cases of **fixed-point** representations: all n bits are used for the integer-part of a number.
- ▶ We could just pretend that the lower $k > 0$ bits are used for a fractional part instead. That is, the bit pattern $a_{n-1} \dots a_k a_{k-1} \dots a_0$ is encoding the binary number $a_{n-1} \dots a_k \cdot a_{k-1} \dots a_0$
- ▶ Equivalently, a number N is represented as the number $N \cdot 2^k$.
- ▶ Sometimes this is the right approach: currency, don't encode kroner or euros, encode ører or euro cents.

How about fractions?

- ▶ Our integer representations are special cases of **fixed-point** representations: all n bits are used for the integer-part of a number.
- ▶ We could just pretend that the lower $k > 0$ bits are used for a fractional part instead. That is, the bit pattern $a_{n-1} \dots a_k a_{k-1} \dots a_0$ is encoding the binary number $a_{n-1} \dots a_k \cdot a_{k-1} \dots a_0$
- ▶ Equivalently, a number N is represented as the number $N \cdot 2^k$.
- ▶ Sometimes this is the right approach: currency, don't encode kroner or euros, encode ører or euro cents.
- ▶ However, how about really large numbers or very small fractions?
E.g., distance to the sun or mass of an electron.
- ▶ And how do we perform arithmetic faithfully to the known precision of our data?

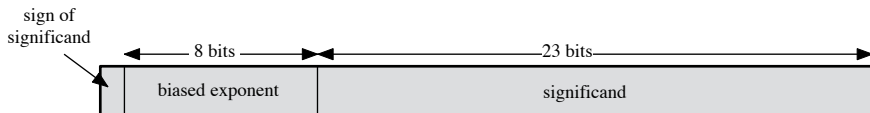
Floating-Point Representations

- ▶ A common format for storing very large numbers and and very small fractions.
- ▶ You have probably seen/used the format already:
 - ▶ Distance to the sun: $1.496 \cdot 10^8$ m
 - ▶ Mass of an electron: $9.109\,383\,56 \cdot 10^{-31}$ kg

Floating-Point Representations

- ▶ A common format for storing very large numbers and and very small fractions.
- ▶ You have probably seen/used the format already:
 - ▶ Distance to the sun: $1.496 \cdot 10^8$ m
 - ▶ Mass of an electron: $9.109\,383\,56 \cdot 10^{-31}$ kg
- ▶ In general, for numbers in base B : $\pm S \cdot B^E$
 - ▶ Sign: plus or minus
 - ▶ Significand: S
 - ▶ Exponent: E (a signed number)
- ▶ We use E to control where the radix-point is placed, hence “floating-point”.
- ▶ We still need to be careful. We have
 - ▶ The **value** of sign, significant, and exponent.
 - ▶ The **representation** of sign, significant, and exponent.

Typical 32-Bit Floating-Point Format



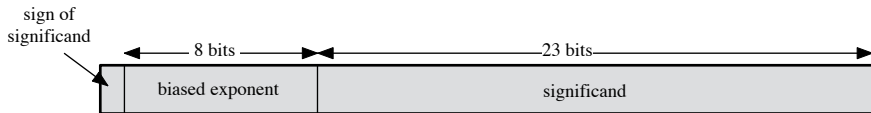
(a) Format

$$\begin{aligned} 1.1010001 \times 2^{10100} &= 0 \ 10010011 \ 101000100000000000000000 = 1.6328125 \times 2^{20} \\ -1.1010001 \times 2^{10100} &= 1 \ 10010011 \ 101000100000000000000000 = -1.6328125 \times 2^{20} \\ 1.1010001 \times 2^{-10100} &= 0 \ 01101011 \ 101000100000000000000000 = 1.6328125 \times 2^{-20} \\ -1.1010001 \times 2^{-10100} &= 1 \ 01101011 \ 101000100000000000000000 = -1.6328125 \times 2^{-20} \end{aligned}$$

(b) Examples

- ▶ Multiple ideas are used:
 - ▶ Sign-magnitude representation: explicit sign bit.
 - ▶ Biased integer representation (see next slide).
 - ▶ Implicit bits in unsigned representation (see later slides).

Typical 32-Bit Floating-Point Format



(a) Format

1	1010001	$\times 2^{10100}$	= 0	10010011	101000100000000000000000	= 1.6328125×2^{20}
-1	1010001	$\times 2^{10100}$	= 1	10010011	101000100000000000000000	= -1.6328125×2^{20}
1	1010001	$\times 2^{-10100}$	= 0	01101011	101000100000000000000000	= 1.6328125×2^{-20}
-1	1010001	$\times 2^{-10100}$	= 1	01101011	101000100000000000000000	= $-1.6328125 \times 2^{-20}$

(b) Examples

- ▶ Multiple ideas are used:
 - ▶ Sign-magnitude representation: explicit sign bit.
 - ▶ Biased integer representation (see next slide).
 - ▶ Implicit bits in unsigned representation (see later slides).

Yet Another Integer Representation: Biased

- ▶ Also called “excess- K ” or “offset” representation.
- ▶ For an integer N and a fixed constant K , represent it N as the unsigned number $N + K$.
- ▶ For example, for 8-bit representations we could use the bias $K = 42$:
 - ▶ Value range: $[-42, 213]$
 - ▶ -42 is encoded as the unsigned number
 $-42 + 42 = 0: 0000\ 0000$
 - ▶ 60 is encoded as the unsigned number
 $60 + 42 = 102: 0110\ 0110$
- ▶ Converting to a value: 1) binary to decimal, 2) subtract the bias.

Yet Another Integer Representation: Biased

- ▶ Also called “excess- K ” or “offset” representation.
- ▶ For an integer N and a fixed constant K , represent it N as the unsigned number $N + K$.
- ▶ For example, for 8-bit representations we could use the bias $K = 42$:
 - ▶ Value range: $[-42, 213]$
 - ▶ -42 is encoded as the unsigned number
 $-42 + 42 = 0$: 0000 0000
 - ▶ 60 is encoded as the unsigned number
 $60 + 42 = 102$: 0110 0110
- ▶ Converting to a value: 1) binary to decimal, 2) subtract the bias.
- ▶ Commonly, for n -bit numbers, we use bias 2^{n-1} or $2^{n-1} - 1$.
E.g., for 8-bit, either 128 or 127.
- ▶ The exponent E will be encoded in biased representation, with $K = 2^{n-1} - 1$
- ▶ (Why biased, why not any of the other integer representations?)

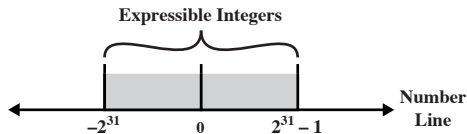
Significand Representation

- ▶ We can write the same number in multiple ways:
 - ▶ $0.110 \cdot 2^5$
 - ▶ $110 \cdot 2^2$
 - ▶ $0.0110 \cdot 2^6$
- ▶ To make operations simpler, we define a canonical way to write it.

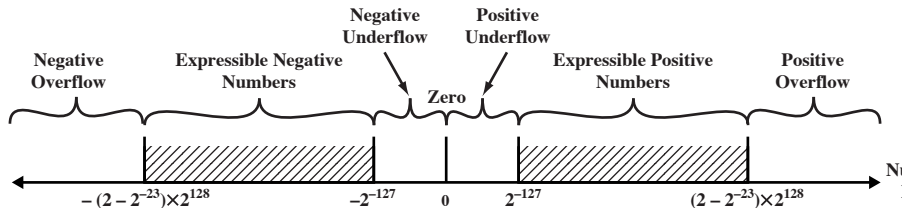
Significand Representation

- ▶ We can write the same number in multiple ways:
 - ▶ $0.110 \cdot 2^5$
 - ▶ $110 \cdot 2^2$
 - ▶ $0.0110 \cdot 2^6$
- ▶ To make operations simpler, we define a canonical way to write it.
- ▶ **Normal form** for non-zero numbers (in binary):
$$\pm S \cdot 2^E, \text{ with } 1 \leq S < 2$$
- ▶ That is, the MSB of S must be 1:
$$\pm 1.bbb \dots b \cdot 2^E$$
- ▶ After arithmetic operations we will try put the result into normal form.
- ▶ Though, this may fail, as E has a finite range (more later).
- ▶ So if the MSB is always 1, do we need to encode it explicitly?
- ▶ No: remove it and we have the **trailing significand**.

Expressible Numbers



(a) Twos Complement Integers



(b) Floating-Point Numbers

- ▶ Floats cover an enormous range of numbers,
- ▶ but not with the same density.

Density of Floating-Point Numbers



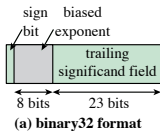
- ▶ Doubling the magnitude:
 - ▶ the exponent is increased by 1,
 - ▶ but the significand has the same number of bits,
 - ▶ so there is the same amount of representable numbers,
 - ▶ but in twice the interval.
- ▶ The closer to zero, the finer the coverage.
The further away, the sparser.
- ▶ Floating-point results are often not exact.
- ▶ Use the right tool for the right job.
 - ▶ Bank accounts: definitely not floats.
 - ▶ Physics simulations: probably floats.
 - ▶ (ECTS points in STADS: ?)

Standard: IEEE 754

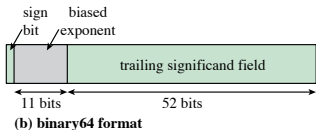
- ▶ Adopted in 1985, revised in 2008.
- ▶ Defines the most important floating-point representations.
- ▶ Also defines the result of operations.
- ▶ Widely adopted and basically all processors follows it.
- ▶ IEEE 754-2008 covers both binary and decimal floating-point representations.
- ▶ We focus on the binary ones.

IEEE 754 Formats

Single



Double



Quadruple

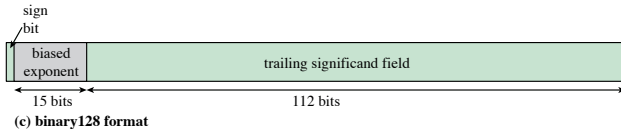


Figure 11.21 IEEE 754 Formats

IEEE 754 Format Parameters

Parameter	Format		
	binary32	binary64	binary128
Storage width (bits)	32	64	128
Exponent width (bits)	8	11	15
Trailing significand width (bits)*	23	52	112
Exponent bias	127	1023	16383
Minimum exponent	-126	-1022	-16382
Maximum exponent	127	1023	16383
Approx. normal number range (base 10)	$[10^{-38}, 10^{38}]$	$[10^{-308}, 10^{308}]$	$[10^{-4932}, 10^{4932}]$
Number of exponents	254	2046	32766
Number of fractions	2^{23}	2^{52}	2^{112}
Number of values	$1.98 \cdot 2^{31}$	$1.99 \cdot 2^{63}$	$1.99 \cdot 2^{128}$
Smallest positive normal number	2^{-126}	2^{-1022}	2^{-16382}
Largest positive normal number	$2^{128} - 2^{104}$	$2^{1024} - 2^{971}$	$2^{16384} - 2^{16271}$
Smallest subnormal magnitude	2^{-149}	2^{-1074}	2^{-16494}

* Does not include implied 1-bit nor sign-bit.

Representation Overview (for 32-Bit Format)

Type	Representation Field			Value
	Sign	Exponent	Fraction	
Positive zero	0	00...0	00...0	0
Negative zero	1	00...0	00...0	-0
Plus infinity	0	11...1	00...0	$+\infty$
Minus infinity	1	11...1	00...0	$-\infty$
Quiet NaN	Either	11...1	1⟨not all 0s⟩	qNaN
Signalling NaN	Either	11...1	0⟨not all 0s⟩	sNaN
Positive normal non-zero	0	⟨e, not all 0s or 1s⟩	⟨f⟩	$1.f \cdot 2^{e-127}$
Negative normal non-zero	1	⟨e, not all 0s or 1s⟩	⟨f⟩	$-1.f \cdot 2^{e-127}$
Positive subnormal	0	00...0	⟨f, not all 0s⟩	$0.f \cdot 2^{-126}$
Negative subnormal	1	00...0	⟨f, not all 0s⟩	$-0.f \cdot 2^{-126}$

- ▶ For formats with other widths the exponent bias changes accordingly.
- ▶ More on subnormal numbers, NaNs, and infinities later.

Additional IEEE 754 Formats

- ▶ Extended precision formats:
 - ▶ Describes how to extend precision of one of the basic formats.
 - ▶ E.g., adding more bits to the significand.
 - ▶ Useful for intermediary results to lessen chance of excessive round-off errors.
 - ▶ The x87 instruction set uses an 80-bit extended format.
- ▶ Extendable precision format:
 - ▶ Describes how to let the user define the extended precision.

- Number Systems
- Computer Arithmetic
- Integers
 - Representations
 - Addition and Subtraction
 - Multiplication
 - Division
- Floating-Point Numbers
 - Representation
 - Arithmetic
- Geometric View on Number Representations

Operations Overview

Given $X = X_S \cdot B^{X_E}$ and $Y = Y_S \cdot B^{Y_E}$:

$$X + Y = (X_S \cdot B^{X_E - Y_E} + Y_S) \cdot B^{Y_E} \quad \text{for } X_E \leq Y_E$$

$$X - Y = (X_S \cdot B^{X_E - Y_E} - Y_S) \cdot B^{Y_E} \quad \text{for } X_E \leq Y_E$$

$$X \cdot Y = (X_S \cdot Y_S) \cdot B^{X_E + Y_E}$$

$$\frac{X}{Y} = \left(\frac{X_S}{Y_S} \right) \cdot B^{X_E - Y_E}$$

- ▶ **Exponent overflow**: a positive exponent exceeds the maximum possible exponent value.
- ▶ **Exponent underflow**: a negative exponent is less than the minimum possible exponent value.
- ▶ **Significand underflow**: when aligning significands, digits flow off the right end of the significand.
- ▶ **Significand overflow**: the addition of two significands of the same sign results in a carry out of the MSB.

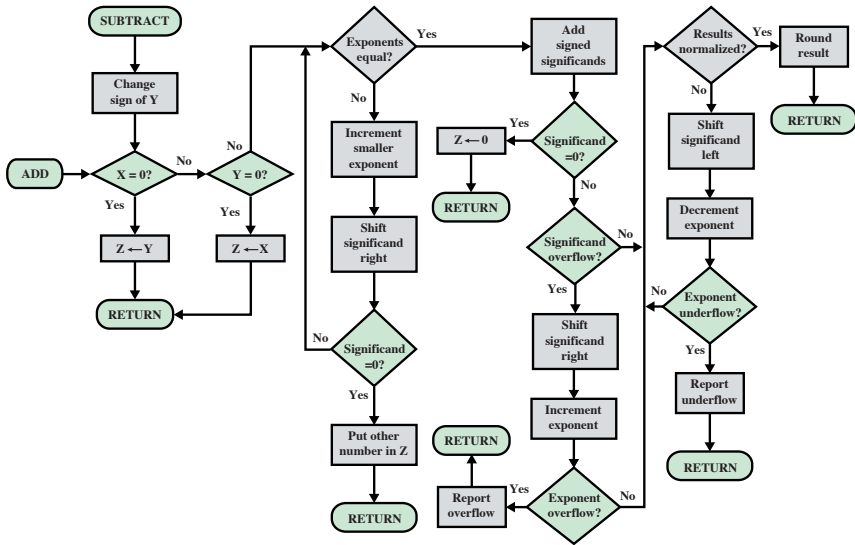
Examples

- ▶ $X = 0.3 \cdot 10^2 = 30$
- ▶ $Y = 0.2 \cdot 10^3 = 200$
- ▶ $X + Y = (0.3 \cdot 10^{2-3} + 0.2) \cdot 10^3 = 0.23 \cdot 10^3 = 230$
- ▶ $X - Y = (0.3 \cdot 10^{2-3} - 0.2) \cdot 10^3 = -0.17 \cdot 10^3 = -170$
- ▶ $X \cdot Y = (0.3 \cdot 0.2) \cdot 10^{2+3} = 0.06 \cdot 10^5 = 6000$
- ▶ $\frac{X}{Y} = \left(\frac{0.3}{0.2}\right) \cdot 10^{2-3} = 1.5 \cdot 10^{-1} = 0.15$

Addition and Subtraction

- ▶ More complicated than multiplication and division.
- ▶ Significands must be aligned first.
- ▶ Algorithm overview:
 1. If we have an implicit significand bit, make it explicit.
 2. If we are subtracting, change the sign of the subtrahend.
 3. Check for zeroes.
 4. Align significands, while adjusting exponents:
 - ▶ This may lose precision,
 - ▶ so align the number with the smallest exponent.
 - ▶ Thus, the bits with the least significance are lost.
 5. Add the signed significands. E.g.,
 - 5.1 Convert from sign-magnitude to two's complement.
 - 5.2 Perform two's complement addition.
 - 5.3 Convert back to sign-magnitude.
 6. Normalize the result.

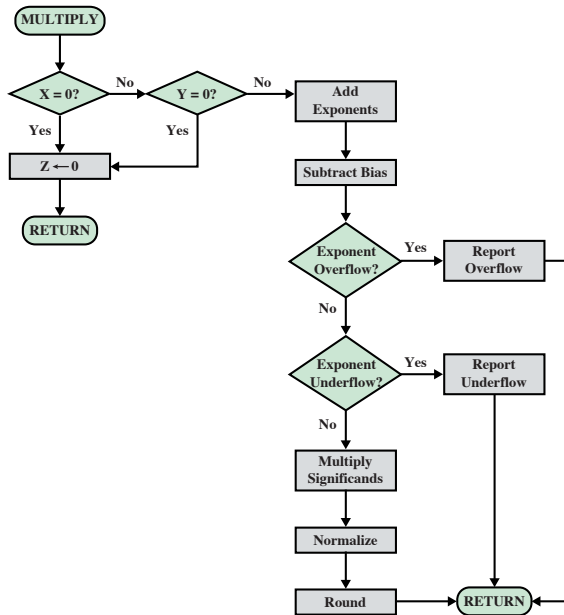
Flowchart for Addition and Subtraction



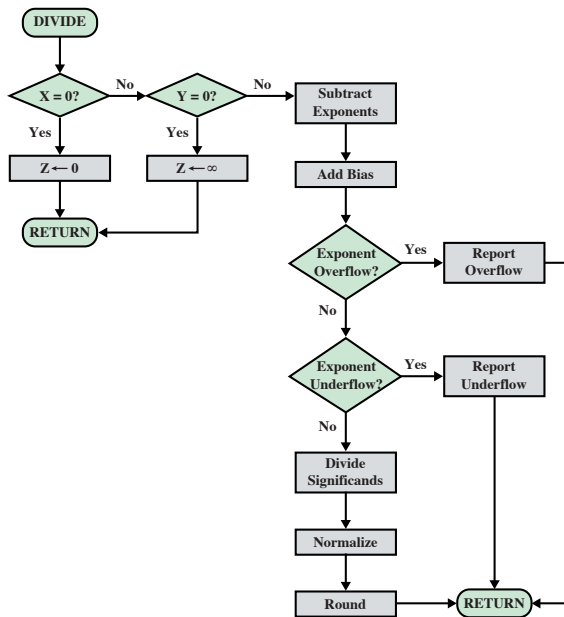
Multiplication and Division

- ▶ Algorithm overview:
 1. Check for zeroes.
 2. Add/subtract exponents.
 3. Multiply/divide significands
(after making implicit bits explicit).
 4. Normalize and round.
- ▶ However, remember that an exponent E is encoded in bias representation.
- ▶ So we do not have X_E and Y_E available, we only have $(X_E + K)$ and $(Y_E + K)$.
- ▶ Addition: $(X_E + K) + (Y_E + K) = (X_E + Y_E) + 2K$
We must subtract K to get the correct result: $(X_E + Y_E) + K$
- ▶ Subtraction: $(X_E + K) - (Y_E + K) = (X_E - Y_E)$
We must add K to get the correct result: $(X_E - Y_E) + K$

Flowchart for Multiplication



Flowchart for Division



Precision Considerations: Guard Bits

Consider the subtraction of very close numbers:

$$x = 1.000\dots00 \cdot 2^1$$

$$-y = 1.111\dots11 \cdot 2^0$$

$$x = 1.000\dots00 \cdot 2^1$$

$$-y = 0.111\dots11 \cdot 2^1 \quad \text{Align significand}$$

$$x - y = 0.000\dots01 \cdot 2^1 \quad \text{Perform subtraction}$$

$$= 1.000\dots00 \cdot 2^{-22} \quad \text{Normalize result}$$

Precision Considerations: Guard Bits

Consider the subtraction of very close numbers:

$$\begin{array}{rcl} x & = & 1.000\dots00 \cdot 2^1 \\ -y & = & 1.111\dots11 \cdot 2^0 \\ \hline x & = & 1.000\dots00 \cdot 2^1 \\ -y & = & 0.111\dots11 \cdot 2^1 & \text{Align significand} \\ \hline x - y & = & 0.000\dots01 \cdot 2^1 & \text{Perform subtraction} \\ & = & 1.000\dots00 \cdot 2^{-22} & \text{Normalize result} \end{array}$$

But what if we internally had a few extra **guard bits** during the calculation:

$$\begin{array}{rcl} x & = & 1.000\dots00 \text{ 0000} \cdot 2^1 \\ -y & = & 1.111\dots11 \text{ 0000} \cdot 2^0 \\ \hline x & = & 1.000\dots00 \text{ 0000} \cdot 2^1 \\ -y & = & 0.111\dots11 \text{ 1000} \cdot 2^1 & \text{Align significand} \\ \hline x - y & = & 0.000\dots00 \text{ 1000} \cdot 2^1 & \text{Perform subtraction} \\ & = & 1.000\dots00 \text{ 0000} \cdot 2^{-23} & \text{Normalize result} \end{array}$$

A factor 2 difference in the result!

Precision Considerations: Rounding

After normalizing we may still have non-zero guard bits.

- ▶ **Round to nearest:**

- ▶ If they are $> 100\dots0$, round up (add 1 to significand)
- ▶ if they are $< 100\dots0$, round down (truncate the extra bits)
- ▶ exactly $100\dots0$?
 - ▶ always up/down: consistent bias
 - ▶ randomly: no bias, but not reproducible
 - ▶ round to even (IEEE):
 - if LSB is 1, round up, else round down (truncate)

Precision Considerations: Rounding

After normalizing we may still have non-zero guard bits.

- ▶ **Round to nearest:**

- ▶ If they are $> 100\dots0$, round up (add 1 to significand)
- ▶ if they are $< 100\dots0$, round down (truncate the extra bits)
- ▶ exactly $100\dots0$?
 - ▶ always up/down: consistent bias
 - ▶ randomly: no bias, but not reproducible
 - ▶ round to even (IEEE):
 - if LSB is 1, round up, else round down (truncate)

- ▶ **Round toward $+\infty$ or toward $-\infty$:** consistent bias, useful to estimate errors: compute twice, with each rounding policy.
- ▶ **Round toward 0:** always truncate the guard bits, very simple, but introduces serious bias.

Arithmetic With Infinities (IEEE)

- ▶ Comparisons satisfy:

$$-\infty < \langle \text{every finite number} \rangle < +\infty$$

- ▶ Operations as normal math, e.g.:

$$5 + (+\infty) = +\infty \qquad (+\infty) + (+\infty) = +\infty$$

$$5 - (+\infty) = -\infty \qquad (-\infty) + (-\infty) = -\infty$$

$$5 + (-\infty) = -\infty \qquad (-\infty) - (+\infty) = -\infty$$

$$5 - (-\infty) = +\infty \qquad (+\infty) - (-\infty) = +\infty$$

$$5 \cdot (+\infty) = +\infty$$

$$5 / (+\infty) = +0$$

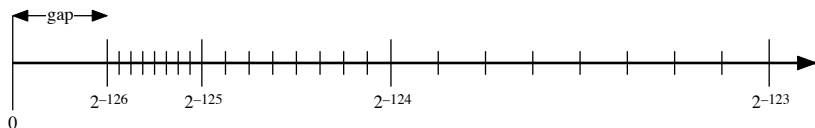
- ▶ (See next slide for the other combinations)

NaNs (IEEE)

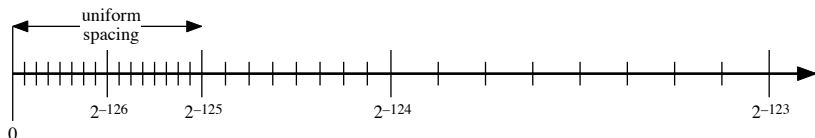
- ▶ **Quiet NaN**: silently propagates through operations.
- ▶ **Signalling NaN**: triggers an exception when used as an operand.
- ▶ The significand bits may contain information on why it was created.

Operation	Quiet NaN produced by
Any	Any operation on a signalling NaN
Addition, subtraction	$(+\infty) + (-\infty)$
	$(-\infty) + (+\infty)$
	$(+\infty) - (+\infty)$
	$(-\infty) - (-\infty)$
Multiplication	$0 \cdot \infty$
Division	$\frac{0}{0}$ or $\frac{\infty}{\infty}$
Remainder	$x \text{ REM } 0$ or $\infty \text{ REM } y$
Square root	$\sqrt{x}$ for $x < 0$

Subnormal Numbers (IEEE)



(a) 32-bit format without subnormal numbers



(b) 32-bit format with subnormal numbers

- ▶ Instead of insisting on the implied 1-bit always.
- ▶ Without subnormal numbers, the underflow gap is large.
- ▶ With subnormal numbers, the underflow gap is comparable to ordinary rounding errors.

- Number Systems
- Computer Arithmetic
- Integers
 - Representations
 - Addition and Subtraction
 - Multiplication
 - Division
- Floating-Point Numbers
 - Representation
 - Arithmetic
- Geometric View on Number Representations

Geometric Depiction of Numbers in Twos Complement

